When Evolutionary Routing Fails: Action Complementarity and Utility Prediction in Structured Decision Models

Anonymous ACL submission

Abstract

Structured decision models such as Jev turn a language-model call into a typed probabilistic judgment: a binary NOUL decision, a candidate CHOICE, or a SCORE. This interface makes inference-time routing attractive: ask a cheap NOUL question first and buy a second, differently framed CHOICE view only when the first answer looks uncertain. We test that intuition with frozen open models. Using Laya 0.4.16 and Vela 2.0 0.3B on CodeXGLUE defect detection, FinDVer claim verification and ProcessBench reasoning-error localisation, we compare a genetic algorithm, an evolution strategy and NSGA-II with fixed policies and budget-matched random search, and measure per-example action complementarity and the predictability of the second action's signed incremental utility. Gross repair opportunities exist: a per-row oracle gains up to 23.3 accuracy points over the better fixed action. Net repair is sparse and pool-dependent: only one of four confirm cells has a positive accuracy interval, and its macro-F1 interval still spans zero. Out-of-fold rank correlation for a signed-utility predictor is weak ($|\text{Spearman}| \leq 0.19$; every group-bootstrap interval includes zero), and confirm cost-adjusted utility is non-positive in all four cells. No genetic-algorithm genome beats the better fixed action, including on a one-shot locked test; the only locked-test gain is over the worse fixed switch, where it learns to stay on NOUL. Calibration improves probabilities but does not create routing signal, because monotone recalibration leaves the band policy class unchanged.

1 Introduction

A claims analyst asks a model whether a sentence is supported by a financial report; a security engineer asks whether a C function contains a vulnerability; a mathematician asks where the first invalid step occurs. Each case needs a decision that software can branch on without a second interpretation step. System One models answer that demand directly. Given a state and a typed question, Jev returns a binary NOUL, a candidate CHOICE, or an ordered SCORE with probabilities (TypeSafe AI, 2026; Deußer et al., 2026); Laya and Vela expose similar interfaces in open weights (Laya Project, 2026; vllm-sr, 2026).

The interface suggests a simple policy. Ask the cheap binary question first, read the returned probability, and spend a second call on a differently framed CHOICE question only when the first answer looks uncertain. The policy is easy to deploy and matches how practitioners use confidence thresholds. It also assumes that the probability of the current answer predicts the value of a different answer. That assumption does not follow from the training objective. RLCD, the recipe behind Jev and the one Laya documents, rewards honest probabilities for the question the model answered (TypeSafe AI, 2026; Laya Project, 2026; Gneiting and Raftery, 2007). A calibrated head can therefore coexist with a router that has no useful signal. Cascades face the same limit: confidence-based deferral is optimal only when downstream errors are tied to upstream confidence, and specialised errors, label noise or distribution shift can break that link (Jitkrittum et al., 2023).

We measure the distinction. Two structured decision models, Laya 0.4.16 and Vela 2.0 0.3B, are frozen, and routing is studied on CodeXGLUE defect detection (Lu et al., 2021), FinDVer claim verification (Zhao et al., 2024) and ProcessBench reasoning-error localisation (Zheng et al., 2025). On the binary tasks the policy pays for NOUL and may pay again for CHOICE; on ProcessBench it decides whether to answer or abstain. We first measure action complementarity, then ask whether the signed incremental value of CHOICE is predictable from the first call, and finally compare a genetic algorithm, an evolution strategy and NSGA-II with fixed policies and budget-matched random search.

A one-shot locked test on official test rows checks the frozen policies.

Gross complementarity is real: on CodeXGLUE, Laya gains 23.3 accuracy points from a per-row oracle over the better fixed action, and CHOICE repairs 14 NOUL errors. The same action breaks 15 correct answers, so the net count on the training pool is -1, and held-out pools do not stabilise the sign. A predictor trained on signed incremental utility has weak out-of-fold rank correlation, and its confirm cost-adjusted utility is non-positive in every cell. The genetic algorithm never significantly beats random search; on a one-shot locked test no genome beats the better fixed action, and the only gain is over the worse fixed switch, where it learns to stay on NOUL. Temperature scaling lowers calibration error but leaves the band policy class unchanged, so it cannot create the missing signal.

The paper contributes a diagnostic decomposition of routing value into action complementarity, signed incremental-utility prediction and search efficacy, measured on frozen structured decision models rather than inferred from end-task accuracy. It reports a pre-specified negative result across five seeds, three pools and four task–model cells, and it shows that calibration and action-value prediction dissociate. The practical lesson is to measure net complementarity and the predictability of the alternative action before building a router; a stronger optimiser cannot recover a signal the action set does not contain.

2 Background and Related Work

Structured decision models replace free-form generation with a typed question and a typed answer. Jev, the commercial System One model from TypeSafe, accepts a state and a set of questions and returns NOUL (a binary judgment), CHOICE (a selection among named candidates) or SCORE (an ordered rating) with associated probabilities (TypeSafe AI, 2026). Deußer et al. (2026) benchmark Jev on 37 datasets, and Guo et al. (2026) apply it to alignment failure detection; both treat the model as a general decision interface rather than a text generator. TypeSafe describes the training objective as Reinforcement Learning for Calibrated Decisions (RLCD), which optimises the reported distribution with strictly proper scoring rules so that probabilities carry information beyond the argmax; the full recipe is not public. Gao and Wang (2026) provide an independent open implementation. The name RLCD also appears in Yang et al. (2023) for Reinforcement Learning from Contrast Distillation; the two are unrelated, and we use RLCD only in the calibrated-decision sense.

The two models we run differ in backbone and supervision while sharing the interface. Laya 0.4.16 is a ModernBERT-large encoder with decision-specific heads, about 421M parameters, and supports NOUL, CHOICE and SCORE in a non-autoregressive pass; it is the open model closest to the RLCD description (Laya Project, 2026; Laya Studio, 2026). Vela 2.0 0.3B is a 22-layer ModernBERT encoder with an 8,192-token window; it exposes the same three actions plus SPAN and SET, and was initialised from the Decision-1.0-Kai CHOICE path with task-specific losses and routing data (vllm-sr, 2026; Warner et al., 2024). Vela's card does not call the recipe RLCD, so we treat Laya as an open RLCD model and Vela as an interface-matched comparison with a different training route.

Calibration is the natural first lens on these probabilities. Neural networks are often overconfident, and temperature scaling or other post-hoc maps can reduce expected calibration error without changing the argmax (Guo et al., 2017). Language models can also partially recognise their own errors, which makes confidence a plausible routing feature (Kadavath et al., 2022), and proper scoring rules formalise why a reported probability should be rewarded for honesty rather than sharpness alone (Gneiting and Raftery, 2007). All of this work concerns the probability of the answer the model has already produced, not whether that probability orders the counterfactual value of a different question type. We measure ECE and Brier for current-answer correctness, then measure the same probability's ability to rank rows that the alternative action would repair.

Cascades and routers allocate computation across models or actions. The classical cascade rejects easy examples early and sends hard ones to a stronger classifier (Viola and Jones, 2001); modern systems learn when to call a larger language model (Chen et al., 2024; Aggarwal et al., 2024; Ong et al., 2024), how many candidates to sample (Ding et al., 2025), or how to combine routing and cascading in one objective (Dekoninck et al., 2025). The theoretical caution comes from Jitkrittum et al. (2023): confidence-based deferral is optimal only when downstream errors are sufficiently tied to upstream confidence, and it can fail under specialised

downstream errors, label noise or distribution shift. That warning is central to learning-to-defer work as well (Mozannar and Sontag, 2020; Chuang et al., 2025). Damani et al. (2025) argue that computation should follow expected benefit rather than surface difficulty. Our experiments instantiate this warning in a structured decision model: the second action is a different question to the same frozen network, not a larger model, and its value depends on an error structure that the first probability does not encode. For ProcessBench, selective prediction supplies the abstention framing (Chow, 1970; Geifman and El-Yaniv, 2017; Kamath et al., 2020; Xin et al., 2021).

Evolutionary methods are a reasonable choice when the policy is a small non-differentiable object and the objective is a cached replay. Genetic algorithms, evolution strategies and NSGA-II cover single-objective and Pareto formulations (Deb et al., 2002; Hansen and Ostermeier, 2001; Stanley and Miikkulainen, 2002), and regularised evolution has been used for architecture search (Real et al., 2019; Elsken et al., 2019). In NLP, EvoPrompt and Promptbreeder evolve prompts with language models in the loop (Guo et al., 2024; Fernando et al., 2024). Those systems change what the model sees; we hold the model and template fixed and search a two-parameter gate over two fixed actions.

Our protocol follows the variance-aware reporting practices of Henderson et al. (2018), Bouthillier et al. (2021) and Dodge et al. (2019): we fix the action set, templates and budget before looking at confirm, report every seed-level interval rather than only the best cell, and report a one-shot locked test under a frozen protocol rather than substituting a confirm number for it.

3 Method and Decision Rules

We study a frozen model M and a fixed action set $\mathcal{A}$. An input x is rendered by a task adapter into a state string and a typed question; the model returns a schema-validated decision with a probability. The policy reads only the first call and decides whether to issue a second call. Model weights, question templates and candidate label sets are never modified during search. This isolates inference-time control from fine-tuning the backbone on the routing objective.

For the binary tasks, CodeXGLUE and FinDVer, the first action is NOUL: the model answers a yes/no question and returns $p(x) = P(\text{true} \mid x)$. The second action is CHOICE: the model selects between two named labels. In CodeXGLUE the NOUL question is "Does this function contain a vulnerability?" and the CHOICE options are `secure` and `vulnerable`; in FinDVer they are "Is the claim supported by the report?" and `supported`/`not supported`. The cascade always pays for NOUL first and pays for CHOICE only when the gate opens. A policy is a genome $g_{\text{cas}} = (\ell, h, u)$ with $\ell, h \in [0, 1]$ and $u \in \{0, 1\}$:

$$\pi_g(x) = \begin{cases} \textsc{Choice}(x), & u = 1 \wedge \ell \le p(x) \le h, \\ \textsc{Noul}(x), & \text{otherwise.} \end{cases} \tag{1}$$

The gate uses the positive-class probability rather than symmetric confidence, because a NOUL question has a designated true label. Genome repair clips ℓ and h to $[0, 1]$, swaps them when $\ell > h$, and maps u to $\{0, 1\}$. Two fixed policies are special cases: $u = 0$ is NOUL-only, and $u = 1, \ell = 0, h = 1$ is fixed switch-only, which still pays for the first NOUL call and therefore costs two calls per row.

ProcessBench has a different action space. The model returns a ranked list of candidate steps, and the policy decides whether to answer with the top-1 step or to abstain. A genome $g_{\text{abs}} = (t, v, m)$ with $t, m \in [0, 1]$ and $v \in \{0, 1\}$ uses the top-1 probability $s_1(x)$ and the top-1/top-2 margin $\Delta(x)$:

$$\text{abstain}(x) = \begin{cases} \text{true}, & s_1(x) < t, \\ \text{true}, & v = 1 \wedge \Delta(x) < m, \\ \text{false}, & \text{otherwise.} \end{cases} \tag{2}$$

When the required confidence fields are missing, the evaluator fails closed: the row is forced to answer and is counted as incorrect rather than silently dropped. We report coverage (the fraction of rows answered) and selective accuracy (accuracy on answered rows).

Fitness combines task quality with the cost of the second call:

$$J(g) = Q(g) - \lambda C(g), \tag{3}$$

where Q is macro-F1 on the binary tasks and step accuracy on ProcessBench, $C(g)$ is the mean number of model calls per row minus one, and $\lambda = 0.05$ is fixed before search. Macro-F1 is primary because both binary datasets are class-imbalanced; accuracy is reported alongside it. The first NOUL call is charged to every policy, including NOUL-only, so the cost term isolates the incremental call

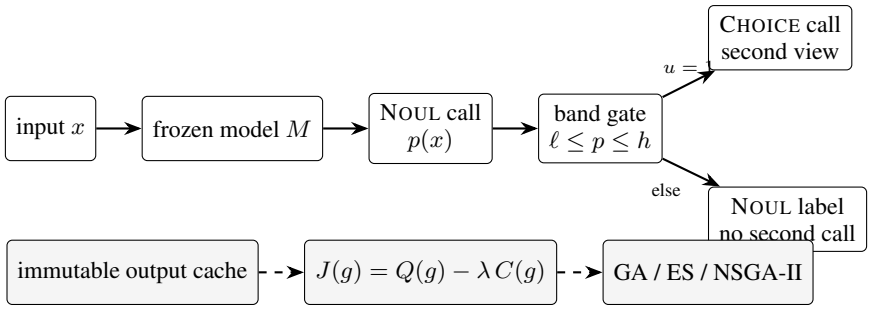


Figure 1: The routing setting. A frozen model answers a first NOUL question; a two-parameter gate decides whether the same model is asked a second CHOICE question. Search operates entirely on cached outputs: a candidate genome is scored by replaying the cache, and the optimiser only sees the scalar fitness J. The dashed path is offline; the solid path is the deployed policy.

rather than the total inference bill. Model outputs are generated once and replayed from an immutable JSON cache, which makes candidate evaluation deterministic and cheap.

The GA uses tournament selection, uniform crossover, Gaussian mutation and bit flips on u. The ES is a $(\mu + \lambda)$ strategy that mutates the two endpoints and fixes $u = 1$. NSGA-II maintains a non-dominated front, optimising accuracy and negative call cost on the binary tasks and selective accuracy and coverage on ProcessBench; because its quality objective is accuracy rather than macro-F1, we keep its intervals separate. The main budget is 24 unique genome evaluations with seeds 17, 23 and 41; a five-seed extension adds 59 and 73. A probe raises the CodeXGLUE budget for Vela to 48. Random search uses the same budget and cache.

One property of the policy class shapes the calibration experiment. The gate depends on $p(x)$ only through the interval $\ell \le p \le h$. Any strictly monotone transformation of p—temperature scaling, for example—maps the interval to another interval and induces the same set of switched rows up to a reparameterisation of the endpoints. A monotone recalibration therefore cannot change the best achievable band policy on a fixed pool. It can move rows between bands, but it cannot create a distinction between repairable and harmful rows that the raw ordering did not already contain. Calibration and routing are therefore evaluated separately.

4 Experimental Design

The experiments use three tasks whose labels and group structure are fixed before any policy is searched. CodeXGLUE defect detection provides 21,854 training, 2,732 dev and 2,732 test functions; the controller pools are drawn only from the official training split, and functions are grouped by an exact hash of their text so that identical functions cannot cross the internal `search_train`, `selection_dev` and `confirm` pools (Lu et al., 2021; Zhou et al., 2019). Each internal pool contains 60 functions. This controls duplicate-function leakage. FinDVer provides financial claims with `supported`/`not supported` labels; its pools are grouped by report, and contain 60, 59 and 44 claims (Zhao et al., 2024). The official FinDVer test labels are not used for controller search. ProcessBench contains 3,400 annotated records over 2,842 mathematical problems; its pools are grouped by problem and contain 60 records each (Zheng et al., 2025; Cobbe et al., 2021; Lightman et al., 2024; Uesato et al., 2022). FinDVer internal pools are carved from official testmini and grouped by report (41/16/16 groups; 60/59/44 claims); labels are used inside these pools for fitting, thresholding and frozen evaluation. The official test split is separate and read only by the locked-test evaluator. State is a `Claim:` line, a blank line, then an `Evidence:` block. CodeXGLUE state is raw function text (positive rates 0.50/0.40/0.45). ProcessBench asks "Which step contains the earliest error?"; its state is the problem plus `[k] step` lines and an `ALL STEPS CORRECT` sentinel.

All inference runs on one Windows machine with CPU-only PyTorch in FP32, two threads and one model resident at a time. Laya is pinned to source revision `b9fe351` and checkpoint revision `7b928d8`; Vela is pinned to revision `d6f03aa`. Peak resident memory stays below 2.2 GB. The native single-action baselines on `search_dev` are as follows: CodeXGLUE accuracy is 0.450/0.650 for Laya (NOUL/CHOICE) and 0.633/0.667 for Vela;

FinDVer is 0.712/0.644 for Laya and 0.458/0.458 for Vela; ProcessBench CHOICE accuracy is 0.10 for both models. The near-identical per-call latency of the two representations (about 2.2–2.8 s) is why call count, rather than a model-specific latency model, is the cost term in Equation 3. The native numbers already contain a warning: the better representation depends on the task and the model, and on FinDVer the two Vela actions are indistinguishable at the aggregate level.

The first experiment measures action complementarity without training a router. For each task–model cell we take the better fixed action on `search_train` and freeze that direction; we then count rows where the alternative action repairs a base error, rows where it breaks a correct base answer, and their difference. The oracle headroom is the accuracy of a per-row oracle minus the accuracy of the better fixed action. Because the direction is fixed on train, the held-out net-repair numbers are not selected on the evaluation pool. We report net repair in percentage points with a group-level percentile bootstrap (5,000 resamples), and pair it with the macro-F1 difference between fixed CHOICE and fixed NOUL on the same rows. ProcessBench has only one evaluated representation, so its headroom is zero by construction and it is excluded from the binary complementarity table.

The second experiment asks whether the signed incremental value of the second action is predictable before that action is executed. The target is $\Delta_i = \mathbf{1}[\text{CHOICE}_i \text{ correct}] - \mathbf{1}[\text{NOUL}_i \text{ correct}] \in \{-1, 0, +1\}$. The features are deployable at the moment of decision: p, $|p - 0.5|$ and the binary entropy $-p \log p - (1-p)\log(1-p)$; no CHOICE output, gold label or downstream feature is used. We compare logistic regression, a shallow decision tree, a single confidence threshold and a never-switch majority baseline. Training uses group-aware three-fold out-of-fold predictions; the predictor and threshold are then chosen together on `selection_dev` by cost-adjusted net utility, frozen, and evaluated once on `confirm`. Net utility is (repairs − regressions) divided by the number of rows; the cost-adjusted variant subtracts $\lambda = 0.05$ per switch. A champion that switches every row is retained, because it reveals whether the predictor adds anything beyond fixed switching.

The search comparison runs optimisers under a matched evaluation budget. GA, ES and NSGA-II are run on the cached pools with 24 unique genome evaluations and seeds 17, 23 and 41; the five-seed extension adds 59 and 73. Random search receives the same budget. The primary deployment comparison is pre-specified: the GA genome selected on `search_train` is frozen and compared with fixed switch-only on `confirm` using paired group-bootstrap macro-F1 intervals. All other cell-level comparisons are exploratory and are reported with effect estimates and intervals rather than as significance claims. We then examine the search trajectory itself: best-so-far fitness per generation, the number of unique genomes actually evaluated, the fraction of single-gene mutations that leave fitness unchanged, the rank correlation between train and dev fitness across the archive, and the effect of raising the budget cap from 24 to 48 on CodeXGLUE with Vela.

Two boundary checks test whether the conclusions generalise. The transfer experiment freezes each source-task genome and applies it to every target task and pool without re-searching; diagonal cells are same-task deployment checks, off-diagonal cells are true transfer. The fault map partitions rows by task, model and a confidence band derived from NOUL probability or candidate margin, and reports the error severity of each cell on discover, dev and confirm. A cell is eligible when it contains at least eight rows on the discover pool, and it is confirmed when the dev or confirm severity is within 0.15 of the discover severity.

Two statistical details matter for reading the tables. All binary comparisons use paired group bootstrap with 5,000 resamples and a fixed random state (Efron and Tibshirani, 1993); the resampling unit is the group (function hash or report), not the row, so dependence within a report is preserved. We report McNemar exact tests for repair/harmful counts (McNemar, 1947) and Wilson intervals for within-cell severity (Wilson, 1927), but we do not call a result significant on the basis of a single seed or a single pool. The locked test was run once under a protocol frozen before any test call. The confirm pools are held out from search and threshold selection but have been inspected during analysis.

5 Results and Discussion

The first question is whether the action set offers anything to route. Table 1 and Figure 2 answer it at the level of individual rows. Gross complementarity is not scarce. On CodeXGLUE with Laya, CHOICE repairs 14 of the 60 NOUL errors, and the per-row oracle reaches 0.783 accuracy against

0.550 for the better fixed action, a headroom of 0.233. The same cell shows why headroom is a poor promise: CHOICE breaks 15 correct NOUL answers, so the net count on the training pool is -1 and the net-repair interval spans zero. The other three binary cells have smaller gross opportunities (headroom 0.067–0.100) and net counts between -6 and $+1$. Across the four cells, the training pool never produces a net repair distinguishable from zero.

Held-out pools do not turn the gross opportunity into a stable net gain. Laya on CodeXGLUE gains 20.0 points on `selection_dev` ($[+1.7, +38.3]$), but the same frozen direction gains only 1.7 points on confirm ($[-18.3, +20.0]$). That gap is a selection-pool effect, not a routing effect: dev is the pool on which the direction was chosen. FinDVer with Laya is the exception that tests the narrative. Its confirm net repair is $+13.6$ points $[+2.0, +24.6]$, and the fixed-action accuracy difference is $+0.136$ $[+0.020, +0.246]$. The corresponding macro-F1 difference, however, is $+0.111$ with an interval that crosses zero, and the better fixed action reverses from NOUL on dev to CHOICE on confirm. The cell therefore contains a real accuracy-level repair signal on confirm, but no macro-F1 gain and no direction a train-selected policy could have exploited. CodeXGLUE with Vela moves the other way: the frozen direction is CHOICE-to-NOUL, the train net is -6, and confirm is exactly zero. FinDVer with Vela is negative on confirm (-2.3 points). Complementarity exists in the action set, but it is sparse, metric-dependent and pool-dependent.

Because macro-F1 is the primary metric, we also compute the fixed-action difference on dev and confirm. CHOICE minus NOUL is $+0.074$ $[-0.093, +0.248]$ and -0.127 $[-0.287, +0.034]$ for Laya on CodeXGLUE; $+0.003$ $[-0.194, +0.197]$ and $+0.078$ $[-0.111, +0.253]$ for Vela on CodeXGLUE; -0.072 $[-0.220, +0.046]$ and $+0.111$ $[-0.007, +0.218]$ for Laya on FinDVer; and -0.106 $[-0.204, +0.021]$ and -0.106 $[-0.217, +0.015]$ for Vela on FinDVer. Every interval includes zero, so no fixed-action macro-F1 difference is significant; the positive accuracy-level net repair on FinDVer with Laya does not become a macro-F1 gain.

The location of the repairs matters as much as their count. Repairs concentrate in the middle and high bands, around $p \in [0.4, 0.8]$, and are close to zero in the lowest band. This is the opposite of what a confidence cascade assumes: a threshold rule switches when the model is uncertain, which is precisely where CHOICE is least likely to fix anything, because the rows it can repair are rows where NOUL was confident and wrong. It also explains why calibration cannot rescue the router: recalibration moves probability mass monotonically, but it does not move repairable rows from the middle of the distribution to the low-confidence tail.

The second question is whether the signed incremental value of CHOICE is predictable from the first call. Table 2 uses $\Delta_i = \mathbf{1}[\text{CHOICE}_i \text{ correct}] - \mathbf{1}[\text{NOUL}_i \text{ correct}] \in \{-1, 0, +1\}$. This is per-example accuracy, not an additive macro-F1 increment. Out-of-fold Spearman correlation between predicted and realised Δ is -0.02, $+0.19$, $+0.13$ and -0.17; every group-bootstrap interval includes zero. Signed-Δ repair AUROC is 0.36, 0.74, 0.46 and 0.40, and regression AUROC 0.61, 0.33, 0.71 and 0.36. The dev-selected thresholds yield confirm raw nets of 0.0, 0.0, 0.0 and -2.3 points, and cost-adjusted nets of 0.0, 0.0, -0.1 and -2.8 points; no cell is positive, and the cost interval for Vela on FinDVer, $[-7.5, -0.1]$, is negative. Vela on CodeXGLUE never switches (0/60). The always-switch baseline is $+1.7$, 0.0, $+13.6$ and -2.3, so it forgoes the $+13.6$ gain on FinDVer with Laya. A repair-target predictor reaches 0.92 repair AUROC but also fails on confirm.

Calibration is the most natural explanation for a failed probability-based router, so it serves as a control. Raw probabilities are poorly calibrated on confirm: ECE ranges from 0.090 to 0.290 and Brier scores (Brier, 1950) from 0.218 to 0.287. Temperature scaling, fitted on `search_train`, reduces ECE in three of four cells (to 0.045–0.169) and Brier in three of four; both rise slightly for Vela on FinDVer. The achievable band class is unchanged, but the selected policies move: raw probabilities give confirm net repairs of -5.0, 0.0, $+2.3$ and -2.3 points, temperature-scaled ones give -1.7, $+1.7$, 0.0 and 0.0, consistent with threshold-selection noise. The repair AUROC makes the point directly: 0.791, 0.444, 0.336 and 0.095 for the four cells; in the last two the probability ranks repairable rows *below* chance.

The third question is whether search converts the available signal into a deployable policy. The comparison uses five seeds, four cells and three pools. GA versus random search is the cleanest test of optimiser value: with five seeds, four task–model cells and three pools, there are 60 paired

Table 1: Action complementarity with the train-selected direction frozen across pools ($n = 60$ per training cell; FinDVer dev/confirm: 59/44). *Headroom* is the accuracy gain of a per-row oracle over the better fixed action on `search_train`; *repair/harmful* counts rows where the alternative fixes or breaks the base decision. Net repair is the paired difference in percentage points (alternative minus base) with a 95% group-bootstrap interval on dev and confirm. Headroom is an upper bound, not a deployable gain.

Task / model	Direction	Headroom	Repair/harmful	Net dev (95% CI)	Net confirm (95% CI)
CodeXGLUE (Laya)	Noul→Choice	0.233	14/15	+20.0 [+1.7, +38.3]	+1.7 [-18.3, +20.0]
CodeXGLUE (Vela)	Choice→Noul	0.067	4/10	-3.3 [-15.0, +8.3]	+0.0 [-15.0, +15.0]
FinDVer (Laya)	Noul→Choice	0.083	5/6	-6.8 [-23.3, +6.0]	+13.6 [+2.0, +24.6]
FinDVer (Vela)	Choice→Noul	0.100	6/7	+0.0 [-9.1, +11.5]	-2.3 [-10.3, +5.0]

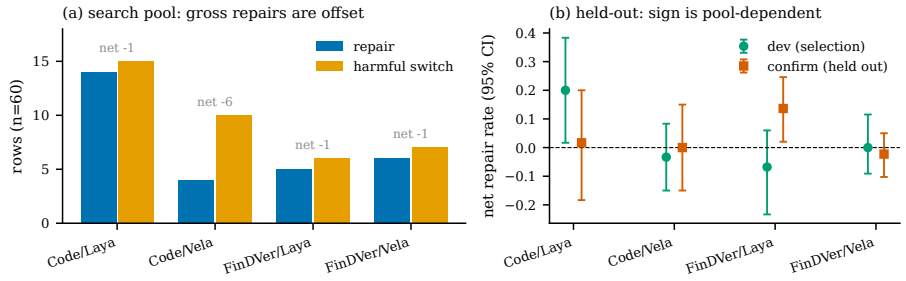


Figure 2: Action complementarity. (a) On the training pool, the second action repairs errors but also breaks correct answers; the net count is printed above each pair. (b) The same train-selected direction applied to dev and confirm: the sign of net repair changes across pools, and only one confirm cell has an interval above zero.

macro-F1 comparisons, and none has an interval that excludes zero. GA versus fixed switch-only has five significant positive cells, all on the FinDVer training pool with Vela and all with an effect near $+0.12$; on dev and confirm, none of the 40 comparisons is significant. The pre-specified deployment comparison—train-selected GA against fixed switch-only on confirm—is null in every seed-level cell. ES versus random search is null in all 30 comparisons; ES versus fixed switching has three significant training cells and one positive seed-level estimate on confirm (CodeXGLUE with Laya, seed 17, $+0.176$). That estimate is not replicated across seeds and is treated as exploratory. NSGA-II deployment is significantly worse than the fixed baseline in five dev cells and one confirm cell, with no positive cell. No optimiser beats the better fixed action at this budget.

Table 3 reports the one-shot locked test. GA beats fixed CHOICE only for Vela on FinDVer ($+0.243$ $[+0.138, +0.318]$), where CHOICE is the worse action (0.413 vs 0.677 for fixed NOUL); the GA stays on NOUL (1.00–1.05 calls/row) and does not beat NOUL (-0.021 $[-0.081, 0.000]$). No GA genome beats the better fixed action in any cell; seeds 23 and 41 show the same pattern.

The trajectory diagnostics explain why more search does not help. In 15 of 21 runs best-so-far fitness does not improve after generation zero; neutral mutations are 0.68–0.73. Train–dev rank correlations are negative on CodeXGLUE (-0.56, -0.51) and positive on FinDVer ($+0.44$, $+0.99$); at budget 48 the GA evaluates only 22, 17 and 23 unique genomes.

Transfer is uniformly unhelpful: six of the 32 source–target units are significantly negative, none positive. The fault map confirms 13/15 cells on dev and 10/15 on confirm; the stable high-severity region is the mid-confidence ProcessBench band.

Table 4 reports the selective baseline. Selective accuracy is 0.133/0.083 at full coverage, 0.100/0.069 at 50%, and 0.105/0.143 at 25%; AURC is 0.894/0.883. Confidence does not order errors well enough for abstention to buy accuracy.

6 Conclusion

A returned probability should decide when a second judgment is worth buying. Our experiments on frozen Laya and Vela across three tasks do not support that promise: evolutionary search produces no advantage over the better fixed action or budget-matched random search.

Gross complementarity is real—a per-row oracle gains up to 23.3 accuracy points—but harmful switches offset repairs, and the net balance is small, metric-dependent and unstable across pools. Cali-

Table 2: Signed incremental-utility prediction. Target $\Delta_i = \mathbf{1}[\text{CHOICE}_i \text{ correct}] - \mathbf{1}[\text{NOUL}_i \text{ correct}] \in \{-1, 0, +1\}$. OOF ρ uses group-aware three-fold CV on `search_train`; the threshold is selected on `search_dev` by cost-adjusted utility and frozen. Raw net is mean Δ; cost-adjusted net subtracts $\lambda = 0.05$ per switch; both are in points with 95% group-bootstrap CIs. Last column: switches/rows.

Task / model	OOF ρ	AUROC repair	AUROC reg.	Dev cost	Confirm raw (95% CI)	Confirm cost (95% CI)	Switches
CodeXGLUE (Laya)	-0.02	0.360	0.607	+17.8	+0.0 [-16.7, +16.7]	-3.5 [-19.9, +12.8]	42/60
CodeXGLUE (Vela)	+0.19	0.744	0.335	+0.0	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0/60
FinDVer (Laya)	+0.13	0.458	0.707	+1.5	+0.0 [+0.0, +0.0]	-0.1 [-0.3, +0.0]	1/44
FinDVer (Vela)	-0.17	0.396	0.361	+2.6	-2.3 [-6.8, +0.0]	-2.8 [-7.5, -0.1]	5/44

Table 3: Locked test: 60 official test rows per cell, templates and GA genomes frozen before any test call. Macro-F1 for fixed NOUL, fixed CHOICE (always switch, two calls) and GA seed 17; seeds 23 and 41 show the same pattern. Paired differences and CIs are in the text.

Cell	NOUL	CHOICE	GA
CodeXGLUE / Laya	0.383	0.375	0.383
CodeXGLUE / Vela	0.422	0.477	0.477
FinDVer / Laya	0.608	0.583	0.583
FinDVer / Vela	0.677	0.413	0.656

Table 4: ProcessBench selective prediction on confirm. Columns are selective accuracy at full coverage, at a 50% target and at a 25% target; parentheses show achieved confirm coverage. Thresholds are selected on `selection_dev` and frozen. AURC is the area under the risk-coverage curve (lower is better).

Model	Full	50% target	25% target	AURC
Laya	0.133 (1.00)	0.100 (0.50)	0.105 (0.32)	0.894
Vela	0.083 (1.00)	0.069 (0.48)	0.143 (0.23)	0.883

bration and action-value prediction dissociate: temperature scaling improves ECE and Brier but not row order, and in both FinDVer cells the probability ranks repairable rows below chance.

Limitations. Pools are small, the NOUL template was not audited, only two open models were run, and the locked test is one frozen run. anonymous.4open.science/r/codex-eacl-8E1E.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. 2024. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems 37*.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Jordan Vasiljevic, César Di Bello, et al. 2021. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3.

Glenn W. Brier. 1950. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

C. K. Chow. 1970. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.

Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki, Xia Hu, and Helen Zhou. 2025. Learning to route LLMs with confidence tokens. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 10859–10878.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. 2025. Learning how hard to think: Input-adaptive allocation of LM computation. In *International Conference on Learning Representations*.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 12987–13010.

Tobias Deußer, Rajkumar Ramamurthy, Lorenz Sparrenberg, and Rafet Sifa. 2026. Evaluating and benchmarking the System One model Jev. *Preprint*, arXiv:2609.37647.

Dujian Ding et al. 2025. BEST-Route: Adaptive LLM routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 13870–13884.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show your work: Improved reporting of experimental results. In

Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pages 2185–2194.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Chrisantha Fernando, Dylan S. Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*.

Zhimin Gao and Pichao Wang. 2026. OpenJev-RLCD: A working RLCD implementation. *Preprint*, arXiv:2609.38850.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*.

Tilmann Gneiting and Adrian E. Raftery. 2007. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 1321–1330.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*.

Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, Lida Zhao, Yutao Wu, Simin Chen, Ying Zhang, and Leo Yu Zhang. 2026. Just ask Jev: Reinforcement learning for calibrated decisions as a zero-shot detector of AI alignment failures. *Preprint*, arXiv:2609.29429.

Nikolaus Hansen and Andreas Ostermeier. 2001. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

Wittawat Jitkrittum, Neha Gupta, Aditya Krishna Menon, Harikrishna Narasimhan, Ankit Singh Rawat, and Sanjiv Kumar. 2023. When does confidence-based cascade deferral suffice? In *Advances in Neural Information Processing Systems 36*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *Preprint*, arXiv:2207.05221.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696.

Laya Project. 2026. Laya: Non-autoregressive System 1 decision engine, version 0.4.16. `https://github.com/NandhaKishorM/laya/tree/b9fe351ce81e2d974bca2bf219c1e4030c0a05ea`. Source revision b9fe351; accessed 2026-11-01.

Laya Studio. 2026. Proper scoring rules: How to reward honest probabilities. `https://laya.studio/learn/proper-scoring-rules`. Accessed 2026-11-01.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS 2021 Datasets and Benchmarks Track*.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Hussein Mozannar and David Sontag. 2020. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *PMLR*.

Isaac Ong, Amjad Almahairi, Vincent Wu, et al. 2024. RouteLLM: Learning to route LLMs with preference data. *Preprint*, arXiv:2406.18665.

Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789.

Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127.

TypeSafe AI. 2026. Introducing System One models and Jev. `https://typesafe.ai/blog/introducing-system-one-models-and-jev`. Accessed 2026-11-01.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *Preprint*, arXiv:2211.14275.

Paul Viola and Michael Jones. 2001. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 511–518.

vllm-sr. 2026. Vela 2.0 0.3b: Model card and training details. `https://huggingface.co/vllm-sr/Vela-2.0-0.3B`. Model revision d6f03aa; accessed 2026-11-01.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. 2024. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *Preprint*, arXiv:2412.13663.

Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Ji Xin, Raphael Tang, Yaoliang Lee, and Jimmy Lin. 2021. The art of abstention: Selective prediction and error conditioning in question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 1040–1051.

Kevin Yang, Dan Klein, Asli Celikyilmaz, Nanyun Peng, and Yuandong Tian. 2023. RLCD: Reinforcement learning from contrast distillation for language model alignment. *Preprint*, arXiv:2307.12950.

Yilun Zhao et al. 2024. FinDVer: Explainable claim verification over long and hybrid-content financial documents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14739–14752.

Chujie Zheng et al. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024.

Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32*.